EMBEDDED SYSTEMS COURSE PROJECT

RTOS Flight Controller

User Manual, Technical Setup Guide, Demo Checklist and Developer Reference for the STM32 / FreeRTOS balance-platform prototype.

TARGET BOARD
STM32 Nucleo-F411RE

CORE STACK
FreeRTOS, STM32 HAL, DMA, UART, I2C, PWM

SENSORS
MPU6050 IMU, BMP180 Barometer

ACTUATORS
Four SG90 Servo Channels

What this guide covers

Hardware wiring, firmware build and flash, RTOS task structure, device-driver layering, GDS data flow, telemetry transport, dashboard operation, ESP32 bridge workflow, demo preparation and troubleshooting.

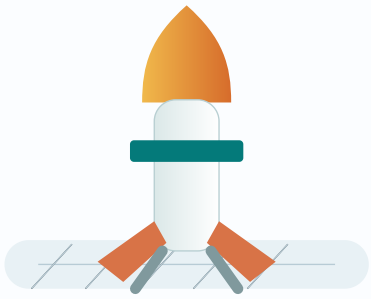

Figure 1. Jury-facing prototype concept: an RTOS-driven balance platform visualized through live telemetry.

Table of Contents

> **Operating principle:** The dashboard is a visualization endpoint only. It must not estimate attitude, infer flight mode or invent sensor/servo values. If a field does not exist in the incoming packet, the corresponding UI value stays at its initial value.

1. Quick Start

Dashboard only

```
cd
Tools/TelemetryDashboard
npm install
npm run dev
```

Open `http://127.0.0.1:5173`.

USB serial telemetry

1. Connect STM32 over USB.
2. Open the dashboard.
3. Click `Serial`.
4. Select the STM32 Virtual COM Port.

WebSocket relay

```
npm run serial:list
npm run serial:bridge -- \
  --
port=/dev/tty.usbmodemXXXX
\
  --baud=115200
```

Recommended demo path: first validate direct USB serial, then validate serial-to-WebSocket relay, and finally move to ESP32 WiFi forwarding. This separates firmware, dashboard and wireless debugging.

2. Project Purpose

This project demonstrates an RTOS-based embedded flight-controller prototype for a rocket-like balance platform. The platform carries sensors and controller software on the upper body, while servo-driven fins or mechanical vanes at the lower section provide visible actuator response.

The system is not a full flight-ready rocket computer. It is a course-scale prototype whose main value is showing the interaction between embedded software layers, physical electronics, real-time task scheduling, sensor acquisition, actuator control, binary telemetry and a jury-facing dashboard.

FreeRTOS STM32 HAL DMA telemetry MPU6050 BMP180 SG90 servos 3D dashboard

3. System Components

Component	Role	Notes
STM32 Nucleo-F411RE	Main controller	Runs FreeRTOS, drivers, control logic and UART telemetry.
MPU6050	IMU	Provides accelerometer and gyroscope data through I2C.
BMP180	Barometer	Provides temperature, pressure and altitude estimate when enabled in firmware telemetry.
SG90 servos	Actuators	Driven by TIM1 PWM channels through a generic servo driver.
ESP32 / relay	Telemetry bridge	Forwards STM32 UART bytes to dashboard over WebSocket.
Telemetry dashboard	Visualization	Parses incoming packets and renders 3D rocket, mission state and sensor panels.

4. Layered Software Architecture

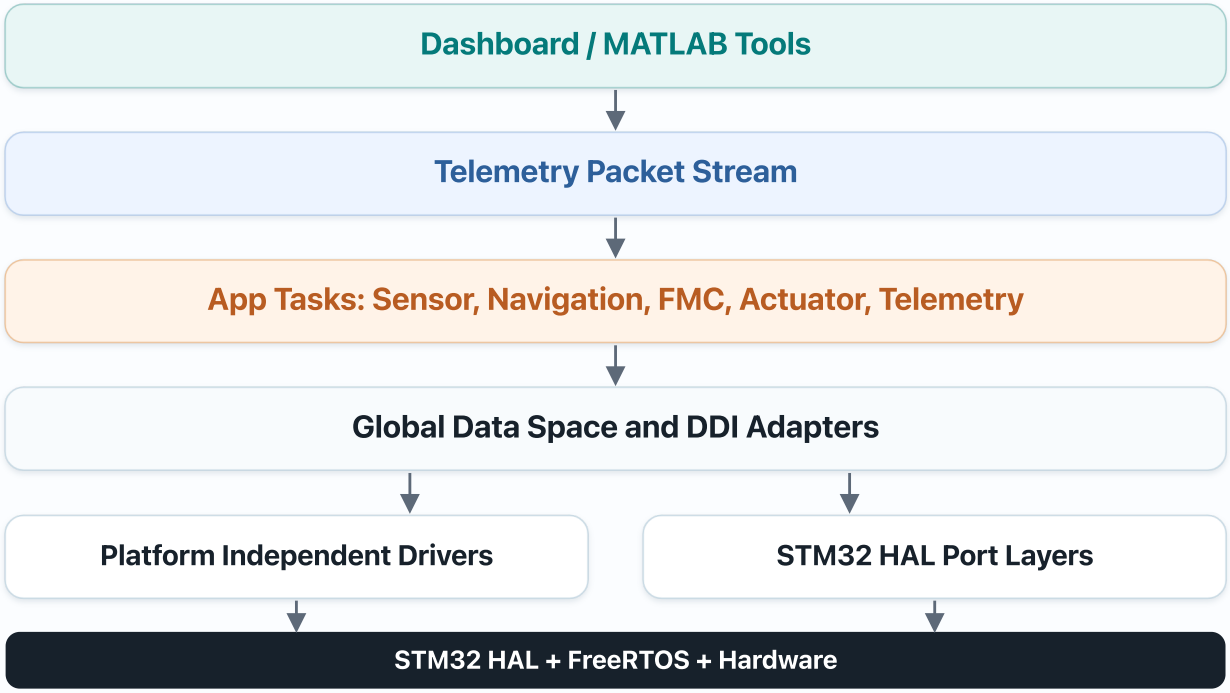

Figure 2. The architecture keeps driver core logic independent from STM32 HAL and RTOS details. Platform-specific actions are injected through port-layer callbacks.

Why this matters

- Core drivers can be compiled and tested on a host machine.
- HAL, DMA, mutex and timing details stay in the STM32 port layer.
- Application tasks exchange data through GDS rather than directly coupling to drivers.

Main code locations

- `Drivers/mpu6050`, `Drivers/bmp180`, `Drivers/servo`
- `App/SensorAcq`, `App/Navigation`, `App/FlightManagementControl`
- `App/Telemetry`, `Platform/STM32`

5. Data Flow

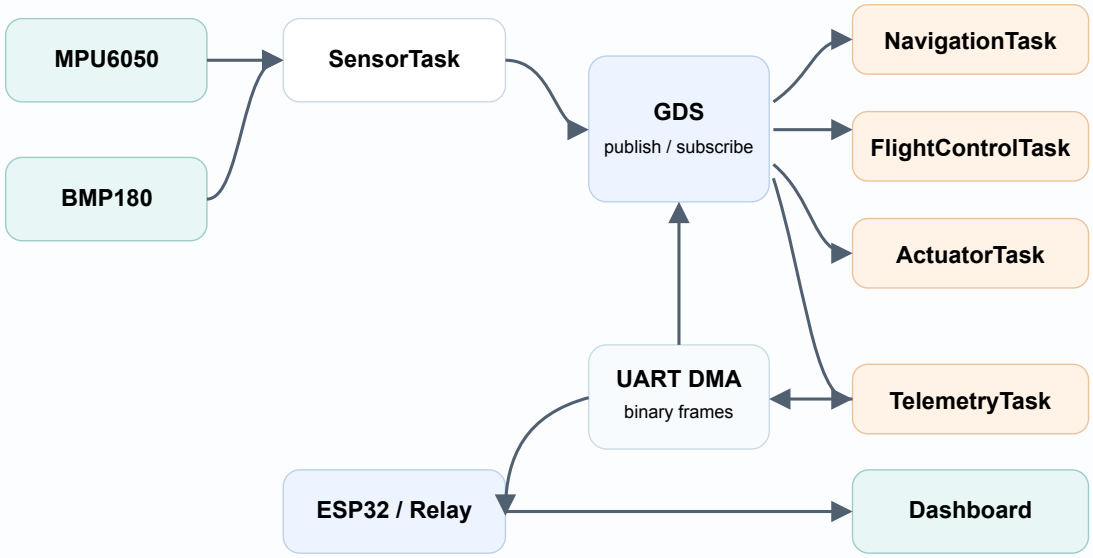

Figure 3. Sensor data is published to GDS, consumed by navigation/control/actuator/telemetry tasks, and finally visualized by the dashboard.

6. RTOS Tasks and Timing

Task	Period	Priority	Responsibility
SensorTask	10 ms	AboveNormal	Runs sensor acquisition, publishes raw IMU and barometer topics.
NavTask	10 ms	AboveNormal	Reads raw IMU, updates vehicle attitude and rates.
FlightControlTask	10 ms	AboveNormal	Runs flight-mode logic and produces fin commands.
ActuatorTask	10 ms	AboveNormal	Applies latest GDS actuator command to servo drivers.
TelemetryTask	50 ms	Normal	Packs GDS data and sends it over UART DMA.

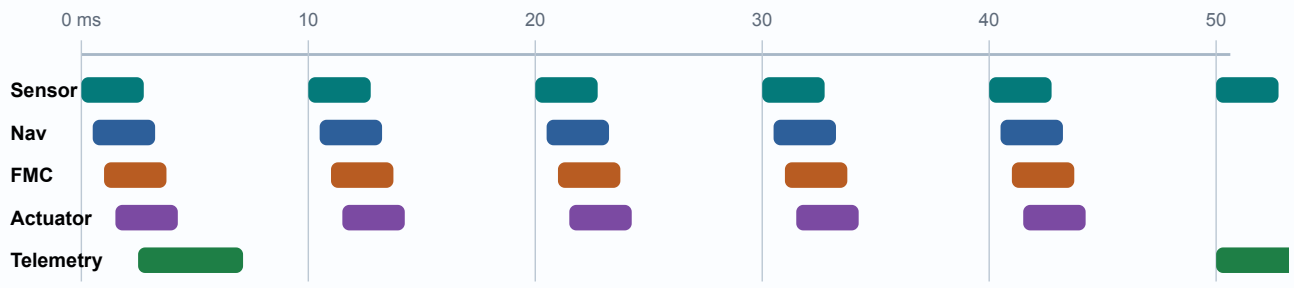

Figure 4. The control loop tasks run at 10 ms; telemetry transmits a lower-rate dashboard stream at 50 ms.

7. Hardware Wiring

I2C sensors

STM32F411RE	Signal	Sensor
PB8	I2C1 SCL	MPU6050 SCL, BMP180 SCL
PB9	I2C1 SDA	MPU6050 SDA, BMP180 SDA
3V3	Power	Sensor VCC
GND	Ground	Sensor GND

Servo PWM

STM32F411RE	TIM channel	Servo
PA8	TIM1_CH1	Fin A
PA9	TIM1_CH2	Fin B
PA10	TIM1_CH3	Fin C
PA11	TIM1_CH4	Fin D

Servo power warning: Do not power SG90 servos from the STM32 3.3 V pin. Use a separate 5 V supply for the servos, and connect servo ground to STM32 ground.

Telemetry UART	Value	Notes
USART2 TX	PA2	STM32 telemetry output, also used by ST-LINK VCP on Nucleo boards.
USART2 RX	PA3	Only needed if bidirectional command input is added.
Baudrate	115200	8 data bits, no parity, 1 stop bit.

8. Firmware Build and Flash

Clone

```
git clone --recurse-submodules https://github.com/AgahEnes/rtos_flight_controller.git
cd rtos_flight_controller
git submodule update --init --recursive
```

CMake build

```
cmake --preset Debug
cmake --build --preset Debug
```

Release build:

```
cmake --preset Release
cmake --build --preset Release
```

Flash

Use STM32CubeIDE or STM32CubeProgrammer with ST-LINK/SWD. Open the generated ELF from `build/Debug` or `build/Release`, download it to the board and reset.

> **Submodule note:** `Drivers/mpu6050` is a submodule. If files are missing, run `git submodule update --init --recursive`.

9. Host Tests

```
cmake -S tests/host -B build/host
cmake --build build/host
ctest --test-dir build/host --output-on-failure
```

Host tests validate application logic and platform-independent driver behavior on the development machine. They do not verify physical wiring, electrical signal quality or sensor presence.

Test area	Purpose
Sensor acquisition	Manager, DDI and GDS publish behavior.
Navigation	Attitude estimator logic and state validity rules.
Flight control	Mode transitions and servo command generation.
Device drivers	MPU6050, BMP180 and servo core behavior without STM32 HAL.

10. Dashboard Operation

Serial mode

1. Start the dashboard.
2. Click `Serial`.
3. Select STM32 Virtual COM Port.
4. Verify frame count increases.

WiFi / WebSocket mode

1. Start ESP32 bridge or relay.
2. Enter `ws://<ip>:<port>/telemetry`.
3. Click `WiFi`.
4. Verify source and packet type.

Dashboard panel	Displayed information	Data source rule
3D Rocket View	Roll, pitch, yaw visualization	Only from parsed attitude fields.
Mission State	Mode, tilt, frame count, packet type	Mode comes from firmware packet, not UI inference.
Sensor Stream	Accel and gyro values	Raw IMU packet fields.
Barometer	Temperature, pressure, altitude	Only updates when packet includes BMP180 fields.
Servo Commands	Fin angles	Only updates when packet includes servo command fields.

11. Telemetry Packet Architecture

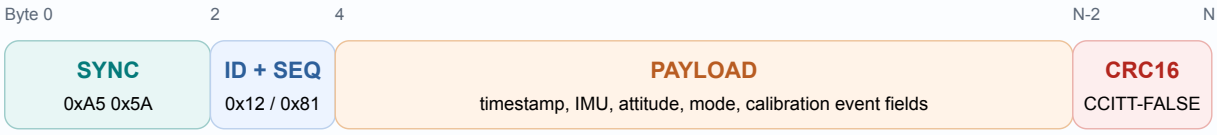

Figure 5. Telemetry packets are binary frames with sync bytes, message id, payload and CRC16/CCITT-FALSE. CRC detects corruption; it is not encryption.

Frame	Message ID	Length	Purpose
IMU + Vehicle State	`0x12`	64 bytes	Main live dashboard frame: timestamp, IMU, attitude, rates, estimator flag and flight mode.
IMU Calibration Event	`0x81`	43 bytes	Asynchronous calibration event frame.
Legacy / Extended frames	`0x10`	Varies	Compatibility with older local tests and mentor packet experiments.

12. ESP32 WiFi Bridge

The ESP32 bridge should forward STM32 telemetry bytes without changing the packet format. The dashboard parser handles frame synchronization, CRC and decoding.

Figure 6. ESP32 is a transport bridge, not a parser. It must not rewrite telemetry payloads.

13. Demo Procedure

1. Show the physical prototype: STM32 board, sensors, servo fins and small mechanical structure.
2. Explain the layered software architecture and GDS publish/subscribe flow.
3. Start the dashboard and connect through USB Serial or WebSocket.
4. Move the platform by hand in roll and pitch; verify the 3D rocket follows the real motion.
5. Show live IMU values and trend graphs.
6. Explain flight mode and telemetry packet format.
7. If servo telemetry is present, show physical fins and dashboard fins moving together.
8. If BMP180 telemetry is present, show the altitude scale and barometer panel.

Pre-demo checklist

- [ ] Repo submodules are initialized.
- [ ] Firmware build passes.
- [ ] STM32 is flashed and reset.
- [ ] Dashboard opens in browser.
- [ ] Frame count increases.
- [ ] Roll/pitch/yaw direction is correct.
- [ ] Servo power and common ground are verified.
- [ ] Backup USB cable and jumper wires are ready.

14. Troubleshooting

Symptom	Likely causes	What to check
Dashboard shows No Link	No serial port, wrong WebSocket URL, bridge not running, firmware not sending.	Use `npm run serial:list`, verify URL and frame count.
Frame count increases but model does not move	Packet has raw IMU but no attitude fields, or NavigationTask is not publishing.	Check packet type and `VehicleState` topic.
Servo does not move	No external 5 V, missing common ground, PWM not started, no active GDS command.	Check PA8-PA11, power rail and `ActuatorCmd`.
I2C sensor fails	SDA/SCL reversed, wrong voltage, missing pull-up, wrong address.	Check PB8/PB9, 3.3 V, addresses `0x68` and `0x77`.
UART/RFD/ESP32 receives nothing	TX/RX not crossed, no common ground, baud mismatch, ST-LINK VCP sharing.	Validate USB serial first, then move to external transport.
Build cannot find toolchain	ARM GNU Toolchain missing from PATH.	Run `arm-none-eabi-gcc --version`.

15. Security and Reliability Notes

Telemetry integrity

CRC16 detects accidental corruption but does not provide authentication or encryption. Final deployments should add HMAC/CMAC and sequence/timestamp replay checks.

Runtime safety

Use stack watermark checks, watchdog heartbeat, stale actuator timeout and neutral servo fallback for safer runtime behavior.

Sensor sanity

Validate NaN/Inf, range and rate-of-change before publishing sensor-derived data to GDS.

Deployment

For final hardware, avoid leaving debug interfaces open and never store production WiFi secrets directly in public source files.

16. Developer Extension Guide

Adding a new sensor

1. Create a platform-independent core driver under `Drivers/<sensor>`.
2. Keep STM32 HAL calls only in the port layer.
3. Add a DDI adapter so SensorManager can use the driver generically.
4. Publish data to a GDS topic.
5. Extend telemetry only after real producer data exists.

6. Update dashboard parser to read explicit packet fields only.

Adding a new telemetry field

```
Producer Task -> GDS Topic -> TelemetryTask Packet Field -> Dashboard Parser -> UI Render
```

> **Do not invert this flow:** The dashboard must not create fake operational data to fill missing firmware fields. It may only display fields that the packet actually contains.

17. Reference Files

Topic	File
Full Markdown manual	`Docs/user_manual_and_setup_guide.md`
Telemetry dashboard	`Tools/TelemetryDashboard/README.md`
Telemetry packet plan	`Docs/Telemetry/telemetry_dashboard_packet_plan.md`
ESP32 bridge notes	`Docs/Telemetry/esp32_wifi_bridge_notes.md`
GDS topics	`App/SensorAcq/global_data_space.h`
Platform RTOS binding	`Platform/STM32/app_platform_port.c`

Generated for the RTOS Flight Controller course project documentation branch. This PDF is a polished print companion to the repository Markdown manual.